CS527 Lab 9 - OpenMP: Tasks, Scheduling and Reductions

Three C++ programs. Each one has a serial version and OpenMP versions of the same computation, checks the parallel results against the serial result, and prints timing tables.

Task	File	What it does
1	`src/task1_mergesort.cpp`	Merge sort with `omp task` / `taskwait`, cutoff experiment
2	`src/task2_schedule.cpp`	`schedule(static)`, `dynamic`, `guided` on uneven work, chunk size experiment
3	`src/task3_evenodd.cpp`	Count even and odd elements: `reduction(+:even, odd)` vs `atomic`

The exact program output that the tables below are taken from is saved in the `results/` folder.

Build and run

```
make            (on Windows with MinGW: mingw32-make)
./task1
./task2
./task3
```

or compile one file by hand:

```
g++ -fopenmp -O2 -Wall src/task1_mergesort.cpp -o task1
```

All three files compile with `-fopenmp -O2 -Wall` without any warning. The programs take no arguments; the sizes and thread counts are written in `main`.

My setup

- CPU: AMD Ryzen 7 5800H, 8 cores / 16 logical CPUs, 16 MB L3 cache
- RAM: 16 GB (2 x 8 GB DDR4-3200)
- OS: Windows 11, compiler: g++ 13.2.0 (MinGW-w64), OpenMP 4.5
- **Thread count method:** `omp_set_num_threads(n)` in the code. Each program uses 1, 2, 4 and 8 threads, as the handout asks (8 = the number of real cores).

How I measured time

- Time is measured with `omp_get_wtime()`, which only changes every 1 ms on my machine. So every measurement is repeated and I report the **average time of one call**. In Task 1 a whole loop of copy + sort is timed

and the time of the copies alone is subtracted, because a single small sort is far shorter than 1 ms.
- The serial version is always run once without timing first (warm-up).
- Speedup S = `T_serial / T_parallel`. Efficiency E = S / threads.
- All numbers are from one run on a laptop that was also running other things, so they are noisy (about +-10-20%, as in Labs 7 and 8).

Task 1 - Parallel Merge Sort with OpenMP Tasks

Implementation

- `merge_sort_serial(a, tmp, left, right)` - normal recursive merge sort on the inclusive range `a[left..right]`.
- `merge(a, tmp, left, mid, right)` - merges `a[left..mid]` and `a[mid+1..right]` into `tmp[left..right]` and copies the result back.
- `merge_sort_parallel` - the same recursion, but the two halves are sorted as two OpenMP tasks, as in the handout:

```
if (right - left < cutoff) { merge_sort_serial(a, tmp, left, right); return; }
int mid = left + (right - left) / 2;
#pragma omp task shared(a, tmp) firstprivate(left, mid)
merge_sort_parallel(a, tmp, left, mid);
#pragma omp task shared(a, tmp) firstprivate(mid, right)
merge_sort_parallel(a, tmp, mid + 1, right);
#pragma omp taskwait
merge(a, tmp, left, mid, right);
```

It is started from `#pragma omp parallel` + `#pragma omp single`: one thread makes the first call, and all threads of the team run the tasks it creates. - `tmp` is allocated **once** in `main`, before sorting, and passed to every call. This is safe because each call only uses `tmp[left..right]` of its own range, and the two child tasks have disjoint ranges. - `a` and `tmp` are `shared` (all tasks work on the same arrays); `left`, `mid`, `right` are `firstprivate`, so each task gets its own copy of the values at the moment the task is created.

Correctness. The serial output is checked to be sorted. Every parallel output is checked to be sorted **and** equal element by element to the serial output: in the main table on 5 separate untimed runs per thread count plus the last timed run, and in the other tables on the last timed run. Input: random integers 0..2^30. **All runs passed.**

Results (cutoff = 2048)

N = 10^5 (126 tasks per sort, serial 8.03 ms)

Threads	Time (ms)	Speedup	Efficiency
1	7.97	1.01	1.01
2	4.79	1.68	0.84
4	3.40	2.37	0.59
8	2.40	3.35	0.42

N = 10^6 (1022 tasks per sort, serial 99.2 ms)

Threads	Time (ms)	Speedup	Efficiency
1	100.2	0.99	0.99
2	55.3	1.79	0.90
4	37.4	2.65	0.66
8	24.7	4.01	0.50

N = 10^7 (16382 tasks per sort, serial 1195.7 ms)

Threads	Time (ms)	Speedup	Efficiency
1	1195.3	1.00	1.00
2	616.3	1.94	0.97
4	422.3	2.83	0.71
8	266.0	4.49	0.56

Small arrays (cutoff = 2048), time in **microseconds**:

Threads	N = 1000 (0 tasks)	N = 10^4 (14 tasks)	N = 5x10^4 (62 tasks)
serial	21.3	657.0	3922.5
1	28.6 (0.75)	675.5 (0.97)	3892.5 (1.01)
2	50.4 (0.42)	450.5 (1.46)	2305.0 (1.70)
4	55.5 (0.38)	360.5 (1.82)	1725.0 (2.27)
8	70.8 (0.30)	266.0 (2.47)	1227.5 (3.20)

Effect of the cutoff, N = 10^6 (serial 109.2 ms in this part of the run):

Cutoff	Tasks per sort	1 thread (ms)	8 threads (ms)	Speedup (8 threads)
1	1999998	374.7	1170.0	0.09
(none)				
16	131070	129.7	102.0	1.07

Cutoff	Tasks per sort	1 thread (ms)	8 threads (ms)	Speedup (8 threads)
256	8190	113.6	30.3	3.60
2048	1022	109.4	27.1	4.03
16384	126	109.3	28.7	3.80
131072	14	108.8	25.0	4.37
524288	2	114.5	61.1	1.79
1000000	0	112.3	113.9	0.96

Questions

What is the difference between OpenMP tasks and `parallel for`? `parallel for` is for a loop whose number of iterations is known when the loop starts: the iterations are divided among the threads, and that is all the parallel work there is. A task is a piece of work (here: "sort this subarray") that a thread creates while it runs, and puts into a pool; any idle thread of the team takes it and runs it. Tasks can create more tasks, so they fit recursive, divide-and-conquer algorithms like merge sort, where the work is a tree and not a flat loop. There is no loop to split in merge sort, so `parallel for` cannot be used for the recursion.

Why is `taskwait` required before merging the two halves? `merge` reads both sorted halves. Creating a task does not run it right away, so without `taskwait` the parent could start merging while the child tasks are still sorting (or have not even started), and it would merge unsorted data. It would also be a data race: the merge writes `tmp[left..right]` and `a[left..right]`, the same memory the children are still using. `taskwait` makes the parent wait until both of its child tasks are finished. Each child itself does a `taskwait` before it finishes, so when the children are done the whole subtree below them is done too.

How does the cutoff threshold affect performance? See the cutoff table. With **no cutoff** (cutoff 1) the program creates 2 million tasks for 10^6 elements, most of them for sorting 2 or 3 numbers. Even on 1 thread this is 3.4x slower than serial (375 vs 109 ms), so one task costs about (375 - 109) ms / 2 million = about 130 ns of overhead, much more than the work in it. With 8 threads it is even worse (1170 ms): all threads keep putting tasks into and taking tasks from the same task queue, which is protected by a lock, so they mostly wait for each other. With a cutoff of **256 to 131072** the number of tasks drops to between a few thousand and 14, the overhead becomes negligible (1 thread is as fast as serial) and the speedup is 3.6-4.4x. If the cutoff is **too large**, there are too few tasks to keep 8 threads busy: with 524288 there are only 2 tasks (2 threads work, 1.79x), and with 10^6 there are none (no speedup at all). So the cutoff should give many more tasks than threads, but each task should still have thousands of elements of work. The handout's 2048 is in the good range.

Why can parallel merge sort become slower than serial merge sort for small arrays? For N = 1000 the cutoff means no task is created at all, but the

program still opens a parallel region, runs `single`, and waits at the barrier at the end. Waking up and synchronizing the threads costs tens of microseconds (Lab 8 measured 13-50 us for one barrier with 2-8 threads on this machine), while the whole serial sort takes only 21 us. So the parallel version is slower (28.6 us with 1 thread, 70.8 us with 8), and more threads make it worse. If the cutoff were small enough to create tasks for such a small array, the 130 ns per task would add even more. From N = 10^4 (14 tasks, 657 us of work) there is enough work for a gain (2.5x with 8 threads).

How does task creation overhead affect scalability? Every task costs time to create, to put in the queue, and to take from it, and the queue is shared by all threads. With a good cutoff this cost is small: for N = 10^7, 16382 tasks x about 130 ns is about 2 ms out of 1195 ms, which is why 1 thread is as fast as serial. With too many tasks (small cutoff) the overhead grows with the number of tasks *and* with the number of threads competing for the queue, so the program gets slower when threads are added (cutoff 1: 375 ms on 1 thread, 1170 ms on 8). Even without overhead the speedup is limited (4.5x on 8 threads for 10^7) because the merges near the top of the tree are serial: the last merge of all 10^7 elements is done by one thread while the other 7 wait, the level below has only 2 merges, then 4, and so on. Merging also mostly moves data, so it is partly limited by memory speed.

Task 2 - Loop Scheduling (static, dynamic, guided)

Implementation

Each element gets the expensive operation from the handout,

```
for (int j = 0; j < work; j++)
    x = x * 1.0000001 + 0.0000001;
```

and the result is written to a second array `b[i]`, so that every run starts from the same input and the results can be compared. The value of `work` depends on the index. Three workloads, all with an average of about 100 inner iterations per element:

Workload	work for element i	Note
uniform	100	every element costs the same
periodic	(i % 1000) / 5, i.e. 0..199	the pattern from the handout
increasing	200 * i / N, i.e. 0..199	the last elements are the most expensive

The handout's `(i % 1000) * 100` means 50000 iterations per element on average. With N = 10^7 one serial run would take about 12 minutes, so I divided the

work by 500. I added the "increasing" workload because the handout's periodic pattern turns out not to be uneven for `static` (see below).

Versions: serial, `schedule(static)`, `schedule(dynamic, 100)` and `schedule(guided)`. For the chunk size experiment a fourth function uses `schedule(runtime)`, and the schedule and chunk size are set with `omp_set_schedule()` before each call.

Correctness. Each parallel version does exactly the same operations on each element as the serial version, so the results must be bit-for-bit equal. Every run was compared with the serial result using `==`: **all matched**.

Results

Time in ms, (speedup, efficiency) in brackets. N = 10^5: average of 50 runs, N = 10^6: 5 runs, N = 10^7: 1 run.

Uniform workload

N = 10^5 (serial 12.72 ms)

Threads	static	dynamic, 100	guided
1	14.22 (0.89, 0.89)	13.40 (0.95, 0.95)	13.00 (0.98, 0.98)
2	7.60 (1.67, 0.84)	7.38 (1.72, 0.86)	7.30 (1.74, 0.87)
4	4.32 (2.94, 0.74)	3.72 (3.42, 0.85)	3.72 (3.42, 0.85)
8	2.48 (5.13, 0.64)	1.92 (6.62, 0.83)	1.94 (6.56, 0.82)

N = 10^6 (serial 143.8 ms)

Threads	static	dynamic, 100	guided
1	153.4 (0.94, 0.94)	142.0 (1.01, 1.01)	137.4 (1.05, 1.05)
2	79.6 (1.81, 0.90)	72.4 (1.99, 0.99)	71.2 (2.02, 1.01)
4	40.6 (3.54, 0.89)	36.2 (3.97, 0.99)	36.0 (3.99, 1.00)
8	21.2 (6.78, 0.85)	18.6 (7.73, 0.97)	18.4 (7.82, 0.98)

N = 10^7 (serial 1448 ms)

Threads	static	dynamic, 100	guided
1	1618 (0.89, 0.89)	1466 (0.99, 0.99)	1447 (1.00, 1.00)
2	796 (1.82, 0.91)	716 (2.02, 1.01)	704 (2.06, 1.03)
4	398 (3.64, 0.91)	364 (3.98, 0.99)	355 (4.08, 1.02)
8	205 (7.06, 0.88)	185 (7.83, 0.98)	181 (8.00, 1.00)

Periodic workload (handout pattern)

N = 10^5 (serial 14.64 ms)

Threads	static	dynamic, 100	guided
1	16.18 (0.90, 0.90)	15.36 (0.95, 0.95)	14.94 (0.98, 0.98)
2	8.44 (1.73, 0.87)	7.54 (1.94, 0.97)	7.76 (1.89, 0.94)
4	4.42 (3.31, 0.83)	3.86 (3.79, 0.95)	4.08 (3.59, 0.90)
8	2.72 (5.38, 0.67)	2.00 (7.32, 0.91)	1.98 (7.39, 0.92)

N = 10^6 (serial 143.4 ms)

Threads	static	dynamic, 100	guided
1	160.2 (0.90, 0.90)	144.4 (0.99, 0.99)	144.4 (0.99, 0.99)
2	82.2 (1.74, 0.87)	73.2 (1.96, 0.98)	74.4 (1.93, 0.96)
4	41.8 (3.43, 0.86)	37.6 (3.81, 0.95)	37.2 (3.85, 0.96)
8	22.0 (6.52, 0.81)	19.0 (7.55, 0.94)	18.8 (7.63, 0.95)

N = 10^7 (serial 1477 ms)

Threads	static	dynamic, 100	guided
1	1627 (0.91, 0.91)	1515 (0.97, 0.97)	1496 (0.99, 0.99)
2	813 (1.82, 0.91)	747 (1.98, 0.99)	735 (2.01, 1.00)
4	405 (3.65, 0.91)	372 (3.97, 0.99)	378 (3.91, 0.98)
8	213 (6.93, 0.87)	193 (7.65, 0.96)	186 (7.94, 0.99)

Increasing workload

N = 10^5 (serial 14.64 ms)

Threads	static	dynamic, 100	guided
1	16.08 (0.91, 0.91)	15.06 (0.97, 0.97)	14.84 (0.99, 0.99)
2	12.66 (1.16, 0.58)	7.56 (1.94, 0.97)	7.54 (1.94, 0.97)
4	7.42 (1.97, 0.49)	3.88 (3.77, 0.94)	3.80 (3.85, 0.96)
8	4.02 (3.64, 0.46)	2.02 (7.25, 0.91)	1.98 (7.39, 0.92)

N = 10^6 (serial 145.6 ms)

Threads	static	dynamic, 100	guided
1	160.6 (0.91, 0.91)	148.6 (0.98, 0.98)	146.6 (0.99, 0.99)
2	125.4 (1.16, 0.58)	76.2 (1.91, 0.96)	74.2 (1.96, 0.98)

Threads	static	dynamic, 100	guided
4	73.2 (1.99, 0.50)	38.2 (3.81, 0.95)	37.4 (3.89, 0.97)
8	39.8 (3.66, 0.46)	19.8 (7.35, 0.92)	19.0 (7.66, 0.96)

N = 10^7 (serial 1444 ms)

Threads	static	dynamic, 100	guided
1	1603 (0.90, 0.90)	1496 (0.97, 0.97)	1459 (0.99, 0.99)
2	1236 (1.17, 0.58)	743 (1.94, 0.97)	740 (1.95, 0.98)
4	721 (2.00, 0.50)	381 (3.79, 0.95)	373 (3.87, 0.97)
8	392 (3.68, 0.46)	196 (7.37, 0.92)	187 (7.72, 0.97)

A note on `static` with 1 thread. In this run `static` is about 10% slower than serial even with 1 thread, in every table, so all its speedups are about 10% lower. This is not caused by scheduling: with 1 thread there is nothing to schedule. I checked the assembly and the inner loop is the same 5 instructions in the serial, static and dynamic versions. In a separate test program with the same functions (N = 10^6, uniform) the difference disappeared (1 thread: static 127 ms, dynamic 133 ms, guided 132 ms), so I think it comes from where the code ends up in memory in this particular build, or from measurement noise. To compare fairly I also give static's speedup against its **own** 1-thread time where it matters below.

Chunk size (N = 10^6, 8 threads)

Time in ms (speedup against serial in brackets). Set with `omp_set_schedule(kind, chunk)` and `schedule(runtime)`.

Uniform workload (serial 144.0 ms)

Chunk	static, chunk	dynamic, chunk	guided, chunk
1	36.0 (4.00)	33.2 (4.34)	18.8 (7.66)
10	20.0 (7.20)	20.0 (7.20)	18.2 (7.91)
100	18.8 (7.66)	18.6 (7.74)	18.2 (7.91)
1000	18.4 (7.83)	18.2 (7.91)	18.2 (7.91)
10000	19.2 (7.50)	19.0 (7.58)	18.8 (7.66)
100000	28.0 (5.14)	28.6 (5.03)	28.0 (5.14)

Increasing workload (serial 145.0 ms)

Chunk	static, chunk	dynamic, chunk	guided, chunk
1	33.0 (4.39)	34.2 (4.24)	19.8 (7.32)
10	21.8 (6.65)	21.2 (6.84)	19.8 (7.32)
100	21.2 (6.84)	20.8 (6.97)	20.2 (7.18)
1000	20.0 (7.25)	20.0 (7.25)	19.8 (7.32)
10000	20.6 (7.04)	20.4 (7.11)	20.6 (7.04)
100000	33.6 (4.32)	33.6 (4.32)	30.4 (4.77)

Questions

What is the difference between static, dynamic, and guided scheduling?

- `static`: the iterations are divided **before the loop starts**. Without a chunk size each thread gets one contiguous block of about N/p iterations (thread 0 the first block, thread 1 the next, ...). With `static, c` the blocks of c iterations are dealt out round-robin. Which thread runs which iteration is fixed in advance and costs nothing at run time, but if some blocks have more work than others, the threads with less work finish early and wait.
- `dynamic, c`: the iterations are handed out **while the loop runs**, c at a time. When a thread finishes its chunk it asks the runtime for the next free chunk. Threads that get cheap chunks simply come back for more, so the work evens out automatically, but every chunk costs a request to a shared counter.
- `guided`: like dynamic, but the chunk size changes: it starts large (about remaining iterations / threads) and gets smaller as the loop goes on, down to c (default 1). Big chunks at the start keep the number of requests low; small chunks at the end even out the finishing times.

Which scheduling strategy performs best for uniform workloads? What about uneven workloads?

Uniform: all three are equally good. Measured against its own 1-thread time, static reaches 1618 / 205 = 7.9x on 8 threads for N = 10^7, the same as dynamic (7.8x) and guided (8.0x). With equal work, equal blocks are already balanced, and static has no run-time cost and gives each thread one contiguous block (good for the cache), so it is the natural choice for uniform work.

Uneven (increasing): static is clearly the worst. With contiguous blocks the last thread gets the most expensive elements. If work grows linearly with i, thread k of p gets a share (2k+1)/p^2 of the total work, so the last thread has (2p-1)/p^2 of it and the best possible speedup is p^2/(2p-1). My measurements (N = 10^7, against static's own 1-thread time 1603 ms) match this closely:

Threads	2	4	8
static, theoretical limit p^2/(2p-1)	1.33	2.29	4.27
static, measured	1.30	2.22	4.09
dynamic, 100 (against serial)	1.94	3.79	7.37
guided (against serial)	1.95	3.87	7.72

Dynamic and guided keep the efficiency above 0.9, so they are about **2x faster than static** on 8 threads. Guided was the best in almost every uneven case: it gets the balance of dynamic with fewer chunk requests.

Periodic (the handout's pattern): static does as well as the others here (N = 10^7: 1627 / 213 = 7.6x against its own 1-thread time). The pattern repeats every 1000 elements, and every static block contains thousands of full periods (N/p = 1.25 million elements for N = 10^7), so every block has the same total work. A workload can be very uneven per element and still be balanced per block; it is only bad for static if the expensive iterations are bunched together, as in the increasing workload.

Why can dynamic scheduling introduce additional overhead? Every time a thread needs a new chunk, it must update a counter shared by all threads (an atomic operation or a lock inside the OpenMP runtime). With small chunks this happens very often: with `dynamic, 1` and N = 10^6 it is a million requests, and with 8 threads they all hit the same counter, whose cache line keeps moving between cores. One element here is only about 150 ns of work, so the request cost is significant: `dynamic, 1` reaches only 4.3x compared to 7.7-7.9x with chunks of 100-1000. Static has none of this cost because the division is calculated once. A second effect of very small chunks is **false sharing**: with chunk 1 neighbouring elements of `b` are written by different threads, and 8 doubles share one 64-byte cache line, so the line keeps moving between the cores. `static, 1` has no run-time overhead at all but is just as slow (36 ms, 4.0x), which shows that false sharing is a big part of the cost here.

How does the chunk size affect performance? Too small (1, 10): high overhead for dynamic and false sharing for static and dynamic (4.0-4.3x with chunk 1). Too large: too few chunks to balance. With chunk 100000 and N = 10^6 there are only 10 chunks for 8 threads, so 2 threads must do 2 chunks while the others do 1: the time is 2/10 of the work instead of 1/8, a limit of 5x, and I measured 5.0-5.1x on the uniform workload. On the increasing workload the last chunks are also the most expensive, which makes it worse (4.3x). In between (100 to 10000) all three schedules are close to 7-8x. Guided is the least sensitive, since its chunk size is only the minimum and the large chunks at the start keep the overhead low.

Task 3 - Counting Even and Odd Elements

Implementation

- **serial** - one loop, `if (a[i] % 2 == 0) even++; else odd++;`
- **reduction** - the handout's version with `reduction(+:even, odd)`
- **atomic** - shared `even` and `odd`, every increment is `#pragma omp atomic`
- **local + atomic** (extra) - each thread counts in private variables and adds them to the shared counters with two `atomic`s at the end, i.e. the reduction written by hand. It shows that `atomic` itself is not the problem, but doing it once per element is.

Input: random integers from -1000000 to 1000000. I included negative numbers on purpose: in C++, `-3 % 2` is `-1`, not `1`. The test `a[i] % 2 == 0` is correct for negative numbers, but a test like `a[i] % 2 == 1` would treat every negative odd number as even.

Correctness.

1. Known-answer test: `a[i] = i - n/2` for n = 10^6 (values -500000 to 499999, exactly half even): serial, reduction, atomic and local + atomic all return even = odd = 500000.
2. For every timed call, the parallel counts were compared with the serial counts and `even + odd == N` was checked. **All matched**, for all sizes and thread counts.

Results

Time in ms. Reduction: (speedup, efficiency). Local + atomic: (speedup). Atomic: how many times **slower** than serial. Reduction and local: average of 10^9/N calls; atomic: average of 100, 10, 1 and 1 calls.

N = 10^5 (serial 0.351 ms)

Threads	reduction	local + atomic	atomic
1	0.362 (0.97, 0.97)	0.365 (0.96)	0.34 (1.0x)
2	0.205 (1.71, 0.86)	0.226 (1.55)	1.54 (4.4x)
4	0.136 (2.58, 0.65)	0.167 (2.09)	1.75 (5.0x)
8	0.133 (2.64, 0.33)	0.167 (2.10)	1.56 (4.4x)

N = 10^6 (serial 3.75 ms)

Threads	reduction	local + atomic	atomic
1	3.80 (0.99, 0.99)	3.71 (1.01)	3.6 (1.0x)
2	1.98 (1.89, 0.95)	2.01 (1.86)	13.3 (3.6x)
4	1.15 (3.26, 0.82)	1.19 (3.16)	14.1 (3.8x)

Threads	reduction	local + atomic	atomic
8	0.65 (5.79, 0.72)	0.75 (4.98)	14.9 (4.0x)

N = 10^7 (serial 39.3 ms)

Threads	reduction	local + atomic	atomic
1	36.9 (1.07, 1.07)	37.5 (1.05)	45 (1.1x)
2	18.9 (2.08, 1.04)	20.1 (1.96)	123 (3.1x)
4	10.5 (3.76, 0.94)	10.4 (3.78)	160 (4.1x)
8	5.9 (6.69, 0.84)	6.1 (6.45)	149 (3.8x)

N = 10^8 (serial 390.4 ms)

Threads	reduction	local + atomic	atomic
1	393.0 (0.99, 0.99)	399.0 (0.98)	397 (1.0x)
2	197.8 (1.97, 0.99)	196.8 (1.98)	1114 (2.9x)
4	97.8 (3.99, 1.00)	98.0 (3.98)	1502 (3.8x)
8	59.7 (6.54, 0.82)	62.2 (6.28)	1641 (4.2x)

Why it scales so well. Counting is a very light operation, so I expected it to be limited by memory like the array sum in Lab 7 (about 2x). Instead `reduction` reaches 6.5-6.7x on 8 threads for the large arrays. The reason is in the assembly of the serial loop: the compiler turns the `if` into a branch on the lowest bit of `a[i]` (`testb $1` + `je`). The numbers are random, so the CPU guesses this branch wrong about half the time, and each wrong guess costs many cycles. One element takes about 3.9 ns, much more than reading 4 bytes from memory. So the loop is limited by the CPU core and not by memory, and 8 cores really do 8 times the work.

Questions

How can multiple variables be included in a reduction clause? List them after the operator, separated by commas: `reduction(+:even, odd)`. Each listed variable gets its own private copy in every thread, initialized to the identity value of the operator (0 for `+`), and at the end of the loop the copies are added into the original variables. If the variables need different operators, use several clauses, for example `reduction(+:sum) reduction(max:largest)`.

Why is the reduction approach generally faster than using atomic updates? With `reduction` each thread counts in its own private variables (which stay in CPU registers) with no synchronization at all, and the private counts are combined only once per thread at the end. With `atomic` every single

element does a locked read-modify-write on a shared counter. With one thread this is cheap (397 vs 390 ms for N = 10^8, because the loop is limited by the branch mispredictions anyway). With more threads, all of them update the same two counters, which also sit next to each other in one cache line, so that cache line has to move from core to core for nearly every element and the threads effectively take turns. The atomic version gets **slower** when threads are added: 3-4x slower than serial and up to 27x slower than `reduction` (N = 10^8, 8 threads: 1641 vs 59.7 ms). The "local + atomic" version is as fast as `reduction` even though it uses `atomic`, because it does only 2 atomics per thread instead of N in total.

Why must the sum of the even and odd counts equal N? Every integer is either even or odd, never both, so in every iteration exactly one of the two counters is increased by exactly 1. After N iterations the two counts must add up to N. If a parallel version gave a smaller sum, increments would have been lost to a race (as in the naive histogram in Lab 8). So `even + odd == N` is a cheap check against lost updates. It does not catch everything, though: a version that put some numbers in the wrong group (for example `a[i] % 2 == 1` with negative numbers) would still add up to N. That is why I also compare with the serial counts and run a test where the right answer is known.

Does increasing the number of threads always improve performance? No.

- For the small array (N = 10^5) the whole job is 0.35 ms, and starting and ending a parallel region costs tens of microseconds. Going from 4 to 8 threads gives almost nothing (0.136 to 0.133 ms), and the efficiency falls to 0.33.
- For `atomic`, more threads make it slower (N = 10^8: 397 ms with 1 thread, 1641 ms with 8), because the threads compete for the same counters.
- For the large arrays with `reduction` it does improve up to 8 threads (6.5x), because this particular loop is limited by the CPU (branch mispredictions) and not by memory. A loop that is limited by memory speed stops improving after about 4 threads (as in Labs 7 and 8).